

Basic (typed function + custom error) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real codebase would actually write it). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You need two things: a custom error, and a function that uses it.

A custom error is just a name for a specific kind of problem. In Python, you make one by writing a small class that is a type of `Exception`.

The function should check its input. If the input is bad (like a negative age), it should raise your custom error instead of continuing. Think about it like this: "if the age is less than 0, something is wrong — stop and say so, using my own error name, not a generic one."

Here are the exact pieces you need:

- To make your own error: `class InvalidAgeError(Exception): pass`
- To trigger it: `raise InvalidAgeError("your message here")`
- To catch it and see the message: `try: ... except InvalidAgeError as e: print(e)`

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The exercise is really about two Python features working together: a custom exception class, and full type hints on a function.

A custom exception is a class that inherits from `Exception`:

```
class InvalidAgeError(Exception):  
    pass
```

`pass` means the class body is empty — you don't need any extra code inside it for a basic custom exception. The class *existing*, with this specific name, is what makes it useful: it lets calling code catch this *one* problem specifically, instead of catching every possible exception with a broad `except Exception`.

The function's type hints should describe its contract precisely:

```
def set_age(age: int) -> None:
```

This tells any reader — without looking at the function body — that it takes an `int` and returns nothing (`None`).

Look specifically at:

- **Exception subclassing:** you're inheriting all of `Exception`'s behavior (the message, the traceback machinery) for free, and only adding a distinct *name* the code can catch.
- **The `raise` statement:** `raise InvalidAgeError(f"age cannot be negative: {age}")` — an f-string here lets you embed the actual bad value in the message.
- **Catching it specifically:** `except InvalidAgeError as e:` — binding the caught exception to `e` lets you access `str(e)` (its message).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think about what a real codebase needs beyond "it raises an error": **who catches this error, and what do they need from it to react properly?** A bare `InvalidAgeError("bad age")` tells a caller *that* something went wrong, but not *what value* was wrong — which matters if the caller wants to log it, retry with a corrected value, or show the user exactly what they typed.

That means the *real* design question isn't just "make a custom exception" — it's "what data should this exception carry so a caller one function away can actually do something useful with it, without re-parsing a string message."

To make the exception actually carry data, you override `__init__`:

```
class InvalidAgeError(Exception):
    def __init__(self, age: int) -> None:
        self.age = age
        super().__init__(f"age cannot be negative: {age}")
```

`self.age = age` stores the bad value on the exception object itself — a caller doing `except InvalidAgeError as e:` can now read `e.age` directly (a real number, ready to use), not just `str(e)` (a sentence they'd have to parse). `super().__init__(...)` still sets the normal human-readable message, so `print(e)` keeps working exactly like before — you're adding a capability, not removing one.

Difference between Basic, Intermediate, and Advanced: Basic gives you the ingredients and exact syntax pieces in plain words. Intermediate explains what each piece of that syntax actually does. Advanced asks the harder design question underneath — not "how do I write a custom exception" but "what should it carry so it's actually useful to whoever catches it" — which is the difference between an exception that merely exists and one that's genuinely well-designed.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make a new error type called InvalidAgeError

make a function called set_age that takes one number (age):
    if age is less than 0:
        stop, and raise InvalidAgeError, with a message about the bad age
    if age is 0 or more, do nothing, it's fine

test it:
    call set_age(25) -> nothing happens, it's valid
    call set_age(-5) inside a try/except -> catch InvalidAgeError, print the mess
```

Here's almost the whole thing — just try running it and reading it line by line:

```
class InvalidAgeError(Exception):
    pass

def set_age(age):
    if age < 0:
        raise InvalidAgeError("age cannot be negative")
```

Expected output if you run just this (nothing calls `set_age` yet): nothing — defining a class and a function doesn't run either of them.

What's missing: the type hints (`age: int` , `-> None`), and the two test calls (one good, one bad, with `try/except`). Add those yourself, then check the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
define:
    class InvalidAgeError(Exception): pass

define:
    def set_age(age: int) -> None:
        if age < 0:
            raise InvalidAgeError(f"age cannot be negative: {age}")
        # implicitly returns None if age is valid – no else branch needed

test:
    set_age(25)                                # runs to completion, no output
    try:
        set_age(-5)
    except InvalidAgeError as e:
        print(f"Caught: {e}")
```

Notice there's no `else` branch needed — if the `if` condition is false, the function just falls through and returns `None` naturally.

```
class InvalidAgeError(Exception):
    pass

def set_age(age: int) -> None:
    if age < 0:
        raise InvalidAgeError(f"age cannot be negative: {age}")
```

What's missing: the two test calls. Write them yourself:

```
set_age(25)  # should run with no output

try:
    set_age(-5)
except InvalidAgeError as e:
    ... # what goes here?
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
define:
    class InvalidAgeError(Exception):
        __init__(age): stores self.age, builds the message via super()

define:
    def set_age(age: int) -> None:
        if age < 0:
            raise InvalidAgeError(age)      # pass the raw value, not a formatted

test, using both pieces of the caught exception:
    try:
        set_age(-5)
    except InvalidAgeError as e:
        print(f"Caught: {e}")              # uses the message
        print(f"Bad value was: {e.age}")   # uses the structured data
        # a real caller might now do: set_age(0) as a safe fallback
```

```
class InvalidAgeError(Exception):
    def __init__(self, age: int) -> None:
        self.age = age
        super().__init__(f"age cannot be negative: {age}")

def set_age(age: int) -> None:
    if age < 0:
        raise InvalidAgeError(age)
```

Fill in the `except` block yourself, this time reading `e.age` as well as `str(e)`, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same underlying idea (check, raise, catch) at 3 completeness levels, shown here both as pseudocode and as near-complete code — Basic proves the concept works, Intermediate adds the type contract real Python expects, Advanced adds the structured-data design that makes the exception genuinely reusable elsewhere in a bigger program.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)